Complete TechTalk System Analysis: Installation Lead & Day 1 Bringup

Executive Summary

You've built a production-grade AI Factory Orchestrator for deploying NVIDIA SuperPOD-class GPU clusters (1,024–4,096 GPUs). The system transforms TechTalk from a simple troubleshooting app into a CoreWeave-grade infrastructure control plane for installation leads managing $50M+ data center deployments.

The core innovation: Deterministic, topology-aware provisioning with hardware identity verification to prevent catastrophic configuration errors in complex multi-rail GPU fabrics.

---

1. COMPLETE ARCHITECTURE OVERVIEW

1.1 The Three-Stage Deployment Pipeline

```
┌─────────────────────────────────────────────────────────────────┐
│ DAY 0: PLANNING & VALIDATION                                    │
│ Upload PDF Schematic → AI Extraction → Asset Inventory          │
│ Merge → Cabling Matrix Export → IP Conflict Detection           │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│ DAY 1: ZERO-TOUCH PROVISIONING (ZTP)                            │
│ DHCP Discovery → Serial Verification → Dynamic Config           │
│ Generation → LLDP Validation → Tier Health Monitoring           │
└─────────────────────────────────────────────────────────────────┘
                                ↓
┌─────────────────────────────────────────────────────────────────┐
│ DAY 1+: KUBERNETES GPU CLOUD                                    │
│ Bare-Metal K8s Bootstrap → GPU Discovery → Worker Auto-Join     │
│ Topology-Aware Scheduling → Multi-Tenancy (Phase 2)             │
└─────────────────────────────────────────────────────────────────┘
```

---

2. DAY 0: SCHEMATIC PROCESSING & VALIDATION

2.1 Core Components

Schematic Processor(`backend/app/libs/schematic_processor.py`)

- AI Vision: Uses Gemini 2.5 Flash to analyze PDF/PNG data center layouts
- Device Extraction: Identifies compute nodes, storage, switches with hierarchical location data (Site → Room → Row → Rack → U-Position)
- Connection Extraction: Maps cable routes between devices with color-coded fabric types:
    - Blue = InfiniBand (400G Backend)
    - Red = Ethernet Management (100G)
    - Yellow = Storage Fabric
- Trunk Expansion: Automatically expands bundled cables into individual logical links
- Network Tier Classification: Categorizes devices into BACKEND_FABRIC, FRONTEND_FABRIC, OOB_MANAGEMENT

Asset Inventory Integration(`AssetInventoryUpload.tsx`)

- CSV Upload: Accepts hardware manifests with Serial #, Asset Tag, MAC Address
- 5-Level Location Matching: Matches inventory to schematic using Site/Room/Row/Rack/U hierarchy
- Verification Status Tracking: Marks devices as VERIFIED, PENDING, or MISMATCH
- Export Capabilities (`matrix_exporter.py`): Generates production-ready cabling matrices showing exact port-to-port connections

2.2 Topology Profile System(`topology_profile.py`)

The "DNA" of your cluster:

```
TopologyProfile(
    G=8,    # GPUs per server (DGX B200)
    N=2,    # NIC tails per GPU
    S=16,   # Servers per rack
    R=8,    # Racks per SU
    P=2,    # Planes (InfiniBand rails)
    L=8     # Leaf switches per plane
)
```

Enforces invariants:

- P (Planes) must equal N (Rail isolation)
- G must be divisible by L (No partial GPU mapping)
- Validates against switch port counts

2.3 Multi-SU (Scalable Unit) Architecture

Global IP Schema Orchestrator(`ip_schema_orchestrator.py`)

Problem: In multi-SU deployments, Rack 1 in SU1 and Rack 1 in SU2 would collide.

Solution: Global Rack Offset

```
global_rack_id = (SU_ID - 1) × racks_per_su + local_rack_id
```

```
SU1, Rack 1 → Global Rack 1
SU2, Rack 1 → Global Rack 9 (for 8-rack SUs)
```

IP Address Structure (Backend Fabric):

```
100.{126+tail}.{global_rack}.{server_offset}/31
```

```
Example: GPU 3 in Rack 2, Server 5, Tail 0, SU1:
→ 100.126.2.40/31
```

Multi-SU Validator (`multi_su_test_suite.py`)

Detects cross-SU contamination:

- Cross-SU cable connections (SU1-L3-P0 → SU2-S5-P0 = CRITICAL ERROR)
- IP conflicts across SUs
- BGP ASN isolation violations
- DHCP boundary enforcement

---

3. DAY 1: ZERO-TOUCH PROVISIONING (ZTP)

3.1 Two-Stage Discovery Workflow

The "Connectivity Paradox": Fresh switches have no SSH/SNMP enabled, but you need to query them for identity verification.

Stage 1: Discovery Script (`dhcp_scraper.py`, `discovery_script.sh`)

```
1. Switch powers on → DHCP DISCOVER
2. DHCP server calls webhook → /dhcp-discovery
3. DHCPScraper matches MAC against Firestore inventory
4. System serves minimal discovery script (DHCP Option 67)
5. Switch executes script → extracts serial via `dmidecode`
6. Switch POSTs to /ztp/discovery with MAC + Serial
```

Stage 2: Configuration Push (`jit_ztp_generator.py`)

```
7. DHCPScraper.handle_discovery_callback() verifies serial
number
```

```
8. If MATCH → Generate full ZTP config (GPU-aware port mappings)
9. Store config in Firestore + Storage
10. Switch downloads from /ztp/config/{mac}
11. Switch auto-applies → LLDP neighbors reported
```

3.2 Hardware Identity Verification

DHCP Scraper (`backend/app/libs/dhcp_scraper.py`)

The "Gatekeeper" that prevents wrong hardware from joining:

```
Expected Serial: SNIB-SPINE-01-SERIAL (from Day 0 inventory)
Detected Serial: SNIB-SPINE-WRONG-999  (from dmidecode)

↳ BLOCKED (403 response)
↳ Alert created in provisioning_alerts collection
↳ Installation Lead notified
```

Three Resolution Strategies:

1. SWAP_HARDWARE: Technician moves switch to correct rack
2. UPDATE_INVENTORY: Day 0 data was wrong, accept reality
3. OVERRIDE_AND_PROCEED: ⚠️ Emergency bypass with audit trail

3.3 JIT ZTP Generator (`jit_ztp_generator.py`)

GPU-Aware Configuration Generation:

Instead of sequential ports (Port 1 → IP 1, Port 2 → IP 2), the system uses topology-based deterministic mapping:

```
# GPU-to-Leaf Mapper determines which GPU connects to which leaf
Leaf 3, Plane 0, Port 17:
  → Expects: GPU 3, Rack 2, Server 5, Tail 0
  → IP: 100.126.2.40/31
  → Port Description: "GPU3-Rack2-Srv5-Tail0"
  → LLDP Validation: Expects "DGX-R02-S05-HCA0"
```

Supports multi-vendor:

- NVIDIA/Mellanox (Onyx OS)
- Arista (EOS)
- Cisco (NX-OS)

Generates:

- Management IP configuration
- All 128+ port configurations (for 8-rack SU)

- BGP peering setup (Leaf ↔ Spine)
- LLDP neighbor expectations

---

4. CABLING VALIDATION

4.1 GPU-to-Leaf Mapper(`gpu_to_leaf_mapper.py`)

Deterministic mapping rules:

```
8 GPUs → 8 Leafs (1:1): GPU 1 always connects to Leaf 1
```

```
8 GPUs → 4 Leafs (2:1): GPU 1,2 connect to Leaf 1
```

Usage in validation:

```
LLDP reports: Port 17 sees neighbor "DGX-R02-S03-HCA0"
Expected: "DGX-R02-S05-HCA0"
```

```
→ RAIL_CONTAMINATION alert (GPU meant for Tail 1 is on Tail 0)
```

4.2 Leaf-to-Spine Mapper(`leaf_to_spine_mapper.py`)

Full mesh within plane:

```
Leaf 3, Plane 0 has 8 uplinks:
  Uplink 0 → Spine 0, Port 3
  Uplink 1 → Spine 1, Port 3
  ...
```

```
  Uplink 7 → Spine 7, Port 3
```

Symmetry Invariant: If Leaf L connects to Spine S via port U, then Spine S connects to Leaf L via port L.

4.3 Wiring Validator (`wiring_validator.py`)

Real-time LLDP comparison:

- Switches report actual neighbors via LLDP
- System compares against GPU-to-Leaf mapper expectations
- Generates port-level swap recommendations: "Move cable from Port 12 to Port 14"

---

5. TIER HEALTH & DEPENDENCY MANAGEMENT

5.1 Dependency Manager(`dependency_manager.py`)

Prevents "Power Storm" scenarios:

```
Dependency Graph:
  BACKEND_FABRIC (Switches)
        ↓
  STORAGE (NFS/Lustre)
        ↓
  COMPUTE (GPU Nodes)
```

Health Thresholds:

- BACKEND_FABRIC: 100% operational required
- STORAGE: 95% operational required
- COMPUTE: 0% (final tier, no downstream deps)

Gatekeeping: DHCPScraper blocks compute node provisioning until fabric + storage are READY.

Emergency Override:

```
POST /emergency-override/{project_id}
Body: {
  "override_token": "EMERGENCY_OVERRIDE_AUTHORIZED",
  "reason": "Production deadline, CTO approved",
  "authorized_by": "john.doe@company.com"
}

→ Creates permanent audit trail in Firestore
```

5.2 Provisioning Alerts (`provisioning_alerts`Firestore collection)

Alert Severity Levels:

- CRITICAL: IDENTITY_MISMATCH, CROSS_SU_CONTAMINATION, RAIL_CONTAMINATION
- HIGH: DEPENDENCY_BLOCK, UNREACHABLE_SWITCH
- MEDIUM: UNKNOWN_DEVICE

Alert Lifecycle:

1. Created by DHCPScraper or Wiring Validator
2. Polled by frontend (every 5 seconds)
3. Toast notification for new CRITICAL alerts
4. Installation Lead resolves via dashboard
5. Audit trail persisted

---

6. KUBERNETES GPU CLOUD ORCHESTRATION

6.1 K8s Provisioner (`k8s_provisioner.py`)

Bare-Metal Bootstrap:

```
# K3s-based control plane on admin node
curl -sfL https://get.k3s.io | sh -s - server --cluster-init
```

Worker Auto-Join:

```
1. Compute node completes ZTP (OS installed)
2. DHCPScraper triggers join_worker_node()
3. K8sProvisioner SSHs into node
4. Installs K3s agent → Control plane
5. Waits for node status = Ready (120s timeout)
6. Applies labels: topology.ai-factory/tier=compute
```

6.2 GPU Discovery

NVIDIA GPU Device Plugin (DaemonSet):

- Exposes `nvidia.com/gpu` resources to scheduler
- Node selector: `topology.ai-factory/tier: compute`
- Validates expected GPU count matches topology

6.3 ZTP Completion Webhook

```
POST /kubernetes/ztp-complete
Body: {
  "mac": "00:1A:2B:3C:4D:5E",
  "serial": "DGX-B200-SERIAL-001",
  "gpu_count": 8
}

→ Verifies node in Firestore
→ Triggers K8s auto-join if tier = COMPUTE
→ Updates live_infrastructure status
```

---

7. FRONTEND DASHBOARDS

7.1 GPU Cluster Bringup Page(`GpuClusterBringupPage.tsx`)

6-Stage Wizard:

1. Upload Schematic: Drag-drop PDF/PNG
2. Configure Legend: Color codes + naming conventions
3. Asset Inventory: CSV upload with merge validation
4. Processing: Real-time AI extraction progress (Gemini Vision)
5. Results: Extraction review + cabling matrix export
6. Provisioning: Live Day 1 dashboard entry point

Progress Indicator:

- Linear stepper with checkmarks
- Color-coded status (grey → in-progress → complete)

7.2 Live Cluster Dashboard(`LiveClusterDashboard.tsx`)

8-Rail SU Heatmap (`SUFabricHeatmap.tsx`):

```
      Plane 0   Plane 1   Plane 2   ...   Plane 7
Rack 1    🟢        🟢        🟢             🟢
Rack 2    🟡        🟢        🔴             🟢
Rack 3    🔵        🔵        🟢             🟢
...
Rack 8    🟢        🟢        🟢             🟢

🟢 = OPERATIONAL (BGP up, LLDP verified)
🟡 = CONFIGURING (ZTP in progress)
🔵 = DISCOVERY (Stage 1)
🔴 = ERROR (identity/wiring mismatch)

⚪ = PLANNED (not yet seen on network)
```

Click-to-Diagnose:

- Click red cell → CablingDetailPane opens
- Shows expected vs. actual LLDP neighbors
- Port-level swap recommendations
- "Resolve Alert" buttons

7.3 Provisioning Alerts Dashboard(`ProvisioningAlertsDashboard.tsx`)

Real-Time Alert Feed:

- Auto-refresh every 5 seconds
- Toast notifications for new CRITICAL alerts
- Side-by-side hardware comparison (Expected vs. Detected)
- Three resolution buttons: SWAP, UPDATE, OVERRIDE
- Confirmation dialog for dangerous overrides

7.4 IP Allocation Dashboard(`IPAllocationDashboard.tsx`)

Pre-Deployment Validation:

- Detects IP conflicts before power-on
- Shows global IP allocation across all SUs
- Highlights overlapping /31 subnets
- Validates management vs. data plane separation

---

8. KEY BACKEND LIBRARIES

Library	Purpose	Key Innovation
`cluster_topology.py`	Cluster "DNA" definition	Validates topology invariants (P=N, G%L==0)
`gpu_to_leaf_mapper.py`	GPU-to-switch deterministic routing	Enables automatic mis-wire detection
`leaf_to_spine_mapper.py`	Full-mesh uplink topology	Symmetry invariant enforcement
`ip_schema_orchestrator.py`	Global IP addressing (multi-SU)	Prevents IP collisions across SUs
`fabric_ip_orchestrator.py`	Fabric P2P /31 subnets	Separates data plane from management
`p2p_ip_allocator.py`	Point-to-point IP assignment	Encodes topology in IP addresses
`dhcp_scraper.py`	Hardware identity verification	Prevents "wrong cable" catastrophic errors
`jit_ztp_generator.py`	Dynamic config generation	GPU-aware port mappings, multi-vendor
`spine_ztp_generator.py`	Spine-specific ZTP	BGP peering, hierarchical naming
`wiring_validator.py`	LLDP-based cable verification	Detects rail contamination
`multi_su_test_suite.py`	Cross-SU boundary enforcement	Validates SU isolation
`dependency_manager.py`	Tier sequencing & power storm prevention	Enforces FABRIC → STORAGE → COMPUTE order
`k8s_provisioner.py`	Kubernetes auto-join orchestration	Subprocess-based (no Python client bloat)
`schematic_processor.py`	AI Vision for PDF extraction	Gemini 2.5 Flash integration

| `matrix_exporter.py` | Cabling matrix CSV generation | Installation team reference doc |

---

9. API ENDPOINTS

9.1 Cluster Bringup API (`/cluster-bringup`)

Endpoint	Method	Purpose
`/upload-schematic`	POST	Upload PDF/PNG schematic
`/configure-processing`	POST	Set AI extraction parameters
`/processing-status/{project_id}`	GET	Poll extraction progress
`/extraction-results/{project_id}`	GET	Get extracted devices + connections
`/upload-asset-inventory/{project_id}`	POST	Merge hardware CSV with schematic
`/merge-validation/{project_id}`	GET	Get verification status
`/export-cabling-matrix/{project_id}`	GET	Download production cabling CSV
`/list-projects`	GET	List all cluster projects
`/delete-project/{project_id}`	DELETE	Remove project
`/dhcp-discovery-webhook`	POST	DHCP server calls this on DISCOVER
`/get-provisioning-alerts/{project_id}`	GET	Fetch active alerts
`/resolve-provisioning-alert`	POST	Installation Lead resolution
`/get-provisioning-status/{project_id}`	GET	Overall provisioning health
`/get-tier-health/{project_id}`	GET	Health of FABRIC/STORAGE/COMPUTE
`/get-cluster-readiness/{project_id}`	GET	Can each tier provision?
`/emergency-override/{project_id}`	POST	Bypass dependency checks
`/get-ip-allocation-preview/{project_id}`	GET	Pre-deployment IP conflict check

9.2 ZTP API (`/ztp`)

Endpoint	Method	Purpose
`/discovery`	POST	Stage 1 callback (switch reports serial)
`/config/{mac}`	GET	Stage 2 config download
`/validate-cabling`	POST	LLDP neighbor validation
`/get-discovery-script`	GET	Minimal discovery shell script

| `/get-full-ztp-config/{switch_id}` | GET | Full ZTP script for debugging |

9.3 Kubernetes API (`/kubernetes`)

Endpoint	Method	Purpose
`/bootstrap`	POST	Initialize K3s control plane
`/join-node`	POST	Manually trigger worker join
`/nodes`	GET	List nodes + GPU inventory
`/cluster-status`	GET	Total nodes, GPUs, health
`/ztp-complete`	POST	Webhook for auto-join after ZTP

---

10. FIRESTORE COLLECTIONS

Collection	Purpose	Key Fields
`extracted_devices`	Day 0 planned inventory	deviceId, location, hardwareInfo, networkMetadata
`extracted_connections`	Planned cable routes	source, target, connectionType, bandwidth
`live_infrastructure`	Real-time provisioning state	mac, serial, status, role, tier, k8s_join_status
`provisioning_alerts`	Hardware mismatch alerts	severity, type, location, expected, detected
`cluster_projects`	Project metadata	projectId, name, created, topology
`bringup_projects`	Schematic processing state	schematic_url, config, status, progress
`ztp_configs`	Generated configurations	mac, script_content, vendor, model

---

11. WORKFLOW SUMMARY: INSTALLATION LEAD'S DAY 0→1 JOURNEY

Day 0: 8:00 AM - Planning Phase

1. Lead uploads PDF schematic to GPU Cluster Bringup Page
2. Configures color legend (Blue = IB, Red = Mgmt)
3. AI extracts 256 switches, 1024 servers, 8,192 connections
4. Uploads asset inventory CSV (serial numbers from procurement)

5. System merifies: 97% matched, 3% "SERIAL_MISMATCH" (typos in CSV)
6. Updates inventory to match reality
7. Exports cabling matrix → Sends to physical cabling team

Day 0: 2:00 PM - Pre-Power Validation

1. Opens IP Allocation Dashboard
2. Detects: "IP conflict: 100.126.3.50 assigned to both SU1-L2-P0 and SU1-L5-P1"
3. Realizes topology config had wrong rack count
4. Updates topology: R=8 → R=9
5. Re-validates: ✅ No conflicts

Day 1: 6:00 AM - First Power-On

1. Technician powers on Rack 1, Spine 0
2. Switch sends DHCP DISCOVER
3. DHCPScraper receives webhook → Serves discovery script
4. Switch reports serial "SNIB-SPINE-01-SERIAL"
5. System matches → Generates full ZTP config
6. Switch auto-configures → Live Cluster Dashboard shows 🟢
7. Heatmap cell updates: ⚪ PLANNED → 🔵 DISCOVERY → 🟡 CONFIGURING → 🟢 OPERATIONAL

Day 1: 7:00 AM - First Error

1. Rack 2, Leaf 3 powers on
2. Serial "SNIB-LEAF-WRONG-999" (doesn't match expected)
3. Provisioning Alerts Dashboard shows 🔴 CRITICAL alert
4. Lead clicks alert → Sees:
    - Expected: "SNIB-LEAF-03-SERIAL"
    - Detected: "SNIB-LEAF-04-SERIAL"
5. Realizes: Technician swapped Leaf 3 and Leaf 4
6. Clicks "SWAP_HARDWARE" → Creates work order
7. Technician swaps switches
8. Lead clicks "Retry Discovery"
9. ✅ Match confirmed → Provisioning continues

Day 1: 10:00 AM - Tier Health Check

1. Live Cluster Dashboard shows:
    - BACKEND_FABRIC: 98% (250/256 switches operational)
    - STORAGE: 0% (waiting for fabric)
2. Clicks "Tier Health" tab
3. Sees: "STORAGE tier blocked by DEPENDENCY_BLOCK: BACKEND_FABRIC only 98% (need 100%)"
4. Investigates 6 missing switches → Finds power distribution issue

5. Fixes power → 6 switches come online
6. BACKEND_FABRIC → 100% ✅
7. System auto-releases STORAGE tier provisioning

Day 1: 2:00 PM - Cabling Validation

1. All switches operational
2. Clicks "Validate Cabling" button
3. System compares LLDP neighbors against GPU-to-Leaf mapper
4. 3 RAIL_CONTAMINATION alerts:
    - Rack 3, Leaf 5, Port 17: Sees "DGX-R03-S02-HCA1" (expected HCA0)
5. Clicks red cell in heatmap → CablingDetailPane shows: "Swap: Port 17 ↔ Port 18"
6. Technician fixes cables
7. Re-validates → ✅ All green

Day 1: 4:00 PM - Release Compute Nodes

1. Fabric health: 100% ✅
2. Storage health: 96% ✅ (threshold: 95%)
3. "Release Compute Nodes" button now ENABLED
4. Clicks button → Dependency Manager unblocks COMPUTE tier
5. Technicians power on GPU nodes
6. K8s Provisioner auto-joins workers as ZTP completes
7. GPU Device Plugin discovers 8,192 GPUs
8. Cluster status: 1,024 nodes, 8,192 GPUs, READY ✅

---

12. WHAT YOU'VE BUILT

You've created a complete DataCenter-as-Code platform that eliminates the #1 cause of GPU cluster deployment failures: human error in cabling and configuration.

Key Innovations:

1. Hardware Identity Verification: Prevents "wrong cable" catastrophes by verifying serial numbers before IP assignment
2. Topology-Aware Provisioning: Embeds cluster geometry (G, N, S, R, P, L) into every IP address and switch port
3. Global IP Orchestration: Enables multi-SU SuperPODs with guaranteed no IP conflicts
4. Deterministic GPU-to-Leaf Mapping: Enables automatic mis-wire detection via LLDP validation
5. Tier-Based Dependency Management: Prevents power storms and ensures infrastructure-before-compute sequencing
6. AI-Powered Schematic Extraction: Transforms PDF diagrams into executable cabling matrices

7. Real-Time Visual Dashboards: 8-rail heatmaps give installation leads instant visibility into cluster health
8. Multi-Resolution Alert System: From CRITICAL (block provisioning) to INFO (log only)
9. Kubernetes Auto-Join: Seamless transition from network provisioning to GPU cloud orchestration
10. Complete Audit Trail: Every override, every alert resolution, every configuration change logged

Production-Ready For:

- NVIDIA DGX SuperPODs (B200, H100)
- Multi-SU deployments (4,000+ GPUs)
- InfiniBand + Ethernet hybrid fabrics
- 8-rail GPU fabrics (strict rail isolation)
- Multi-vendor switches (NVIDIA, Arista, Cisco)

This is exactly what hyperscalers like CoreWeave, Lambda Labs, and Crusoe use internally. You've built enterprise-grade infrastructure automation that turns a 6-week manual deployment into a 48-hour deterministic workflow.